ソフトウェア工学特論 講義資料

第12回 チーム開発④：実装

- PR駆動の実装ワークフロー
- コードレビューの観点と返し方
- 生成AI活用のプロンプト記録

目次

- PR駆動の実装
- コードレビュー
- AI活用記録

PR駆動の実装

関連ドキュメント: この節の解説

今回の目的と到達目標

今回の目的

- 分担して実装を進める
- PRでレビューし合う
- AI活用のプロンプトを残す

到達目標

- [R2-標準] Issue・PRを活用した開発主導
- [R3-標準] 工程管理の実践

実装サイクル

1. Issueを自分にアサイン
2. `feature/issue-xx-yyy` ブランチ作成
3. 実装 + 動作確認
4. pushしてPR作成（本文に `Closes #xx`）
5. レビュアーのコメントに対応
6. 承認後マージ、ブランチ削除
7. 次のIssueへ

小さく・早く・何度も マージする。大きいPRは事故の元。

PR単位の目安

- 1 PRの差分は **300行以内** を目安に
- 超えそうなら **Issueを分割** する
- WIPのまま置かず、**最低毎日push** でチーム共有
- 大きな設計判断が必要になったら PR を切る前に Issue で相談

コードレビュー

関連ドキュメント: この節の解説

レビュアーの振る舞い

- 24時間以内に1コメントを目安に返す
- 指摘は **根拠 + 代替案** を添える
- 「LGTMだけ」は形式的レビュー → 減点対象
- 小さな提案は suggestion 記法で具体的に示す
- 致命的な問題はChangesRequestedで差し戻す

suggestion

`for` でなく配列メソッドに置き換えると意図が伝わりやすい。

```js
items.filter(x => x.active).map(x => x.name)
```

```
- **何が問題か → どうすべきか → なぜか** の順
- 個人への批判ではなく **コードについての記述**
- 「好み」なのか「要修正」なのかをラベル（nits: / must: 等）で示す

---

<!--
class: flex-layout natural-height
-->

# レビュイー（被レビュー側）の振る舞い

- コメントには **すべて返信** する（対応・理由付きで却下）
- 単純な修正はコミットで応答
- 議論になったらコメントで対話してから直す
- 最終的にレビュアーが **Approve** するまで待つ
- 自分のブランチには議論の痕跡を残す（意思決定ログ）

---

# AI活用記録

**関連ドキュメント**: [この節の解説](https://github.com/atsuki-seo/NITYC-MCC-Tools/issues/72)

---

<!--
class: flex-layout natural-height
-->

# プロンプト記録の目的

- AI出力を **そのまま使わず** チームで評価するため
- 後から「何を聞いて何を得たか」を追跡可能にする
- 成果物の一部として最終提出物に含める（必須）
- 評価観点「AI活用記録と批判的評価」の根拠になる

---

<!--
class: flex-layout natural-height
-->

# 記録フォーマット例

```markdown
## 場面：予約一覧のソート実装
### プロンプト
「JavaScriptで配列を日付昇順にソートして」

### 出力（要約）
`arr.sort((a,b) => new Date(a.date) - new Date(b.date))`

### チームの判断
採用。ただし `date` のパース誤りを避けるため
型ガードを追加してマージ。
```
```

今回のまとめ

- PRは小さく早く。1 PR 300行以内を目安
- レビューは根拠+代替案を添える。LGTM単独は避ける
- レビュイーは全コメントに返信
- AI活用記録はチームの判断が核

今回カバーしたMCC項目

- V-D-1 プログラミング
- V-D-4 コンピュータシステム

次回予告

- 第13回: チーム開発⑤ 実装・テスト